

# 大学生迎新引导服务数字人系统项目规约

|                     |    |
|---------------------|----|
| 一、编程规约 .....        | 2  |
| (一)命名风格 .....       | 2  |
| (二)常量定义 .....       | 2  |
| (三)代码格式 .....       | 3  |
| (四)OOP 规约 .....     | 3  |
| (五)集合处理 .....       | 4  |
| (六)并发处理 .....       | 6  |
| (七)控制语句 .....       | 7  |
| (八)注释规约 .....       | 7  |
| 二、异常日志 .....        | 8  |
| (一)异常处理 .....       | 8  |
| (二) 日志规约 .....      | 9  |
| 三、安全规约 .....        | 10 |
| 四、MySQL 数据库 .....   | 11 |
| (一)建表规约 .....       | 11 |
| (二)索引规约 .....       | 12 |
| (三)SQL 语句 .....     | 12 |
| (四)ORM 映射 .....     | 12 |
| 五、工程结构 .....        | 13 |
| 应用分层 .....          | 13 |
| 七、微服务与接口规约 .....    | 14 |
| 八、AI 与大模型应用规约 ..... | 15 |

# 一、编程规约

## (一)命名风格

1. 【强制】代码中的命名均不能以下划线或美元符号开始，也不能以下划线或美元符号结束。
2. 【强制】代码中的命名严禁使用拼音与英文混合的方式，更不允许直接使用中文的方式。
3. 【强制】类名使用大驼峰命名法 UpperCamelCase 风格，但以下情形例外：DO / BO / DTO / VO / AO / PO 等。
4. 【强制】方法名、参数名、成员变量、局部变量都统一使用小驼峰 lowerCamelCase 风格，必须遵从驼峰形式。
5. 【强制】常量命名全部大写，单词间用下划线隔开。
6. 【强制】抽象类命名使用 Abstract 或 Base 开头；异常类命名使用 Exception 结尾；测试类命名以它要测试的类名开始，以 Test 结尾。
7. 【强制】类型与中括号紧挨相连来定义数组。
8. 【强制】POJO 类中布尔类型的变量，都不要加 is 前缀，否则部分框架解析会引起序列化错误。反例：定义为基本数据类型 Boolean isDeleted 的属性，它的方法也是 isDeleted()，RPC 框架在反向解析的时候，“误以为”对应的属性名称是 deleted，导致属性获取不到，进而抛出异常。
9. 【强制】包名统一使用小写，点分隔符之间有且仅有一个自然语义的英语单词。包名统一使用单数形式，但是类名如果有复数含义，类名可以使用复数形式。
10. 【强制】对于 Service 和 DAO 类，基于 SOA 的理念，暴露出来的服务一定是接口，内部的实现类用 Impl 的后缀与接口区别，若接口仅有一个实现类，可不强制拆分接口与实现，但全组必须统一约定。

## (二)常量定义

1. 【强制】不允许任何魔法值（即未经预先定义的常量）直接出现在代码中。
2. 【强制】long 或者 Long 初始赋值时，使用大写的 L，不能是小写的 l，小写容易跟数字 1 混淆，造成误解。

说明：Long a = 2l; 写的是数字的 21，还是 Long 型的 2？

3. 【推荐】如果变量值仅在一个固定范围内变化用 `enum` 类型来定义。

说明：如果存在名称之外的延伸属性使用 `enum` 类型，下面正例中的数字就是延伸信息，表示一年中的第几个季节。

4. 【强制】报到状态（未报到 / 已线上报到 / 已缴费 / 已入住等）、问答类别、缴费渠道、错误码分段等固定取值统一用 `enum` 定义，禁止魔法值散落；状态枚举须带注释说明每个状态的业务含义与流转方向。

### (三)代码格式

1. 【强制】大括号的使用约定。如果是大括号内为空，则简洁地写成`{}`即可，不需要换行；如果是非空代码块则：

- 1) 左大括号前不换行。
- 2) 左大括号后换行。
- 3) 右大括号前换行。
- 4) 右大括号后还有 `else` 等代码则不换行；表示终止的右大括号后必须换行。

2. 【强制】左小括号和字符之间不出现空格；同样，右小括号和字符之间也不出现空格。

3. 【强制】`if/for/while/switch/do` 等保留字与括号之间都必须加空格。

4. 【强制】任何二目、三目运算符的左右两边都需要加一个空格。

5. 【强制】采用 4 个空格缩进，禁止使用 `tab` 字符。

6. 【强制】注释的双斜线与注释内容之间有且仅有一个空格。

7. 【强制】单行字符数限制不超过 120 个，超出需要换行。

8. 【强制】方法参数在定义和传入时，多个参数逗号后边必须加空格。

9. 【强制】IDE 的 `text file encoding` 设置为 `UTF-8`；IDE 中文件的换行符使用 `Unix` 格式，不要使用 `Windows` 格式。

### (四)OOP 规约

1. 【强制】避免通过一个类的对象引用访问此类的静态变量或静态方法，无谓增加编译器解析成本，直接用类名来访问即可。

2. 【强制】所有的覆写方法，必须加 `@Override` 注解。

3. 【强制】相同参数类型，相同业务含义，才可以使用 `Java` 的可变参数，避免使用 `Object`。

4. 【强制】Object 的 equals 方法容易抛空指针异常，应使用常量或确定有值的对象来调用 equals。
5. 【强制】所有的相同类型的包装类对象之间值的比较，全部使用 equals 方法比较
6. 关于基本数据类型与包装数据类型的使用标准如下：
  - 1) 【强制】所有的 POJO 类属性必须使用包装数据类型。
  - 2) 【推荐】所有的局部变量使用基本数据类型。
7. 【强制】定义 DO/DTO/VO 等 POJO 类时，不要设定任何属性默认值。
8. 【强制】POJO 类必须写 toString 方法。使用 IDE 中的工具：source> generate toString 时，如果继承了另一个 POJO 类，注意在前面加一下 super.toString。
9. 【推荐】使用索引访问用 String 的 split 方法得到的数组时，需做最后一个分隔符后有无内容的检查，否则会有抛 IndexOutOfBoundsException 的风险。
10. 【推荐】当一个类有多个构造方法，或者多个同名方法，这些方法应该按顺序放置在一起，便于阅读。
11. 【推荐】setter 方法中，参数名称与类成员变量名称一致，this.成员名 = 参数名。在 getter/setter 方法中，不要增加业务逻辑，增加排查问题的难度。
12. 【推荐】循环体内，字符串的连接方式，使用 StringBuilder 的 append 方法进行扩展。
13. 【推荐】合理使用 final 规则。

## (五)集合处理

1. 【强制】关于 hashCode 和 equals 的处理，遵循如下规则：
  - 1) 只要重写 equals，就必须重写 hashCode。
  - 2) 因为 Set 存储的是不重复的对象，依据 hashCode 和 equals 进行判断，所以 Set 存储的对象必须重写这两个方法。
  - 3) 如果自定义对象作为 Map 的键，那么必须重写 hashCode 和 equals。

说明：String 重写了 hashCode 和 equals 方法，所以我们可以非常愉快地使用 String 对象作为 key 来使用。
2. 【强制】ArrayList 的 subList 结果不可强转成 ArrayList，否则会抛出 ClassCastException 异常，即 java.util.RandomAccessSubList cannot be cast to java.util.ArrayList。说明：subList 返回的是 ArrayList 的内部类 SubList，并不是 ArrayList，而是 ArrayList 的一个视图，对于 SubList 子列表的所有操作最终会反映到原列表上。

3. 【强制】在 subList 场景中，高度注意对原集合元素个数的修改，会导致子列表的遍历、增加、删除均会产生 ConcurrentModificationException 异常。

4. 【强制】使用工具类 Arrays.asList()把数组转换成集合时，不能使用其修改集合相关的方法，它的 add/remove/clear 方法会抛出 UnsupportedOperationException 异常。

说明：asList 的返回对象是一个 Arrays 内部类，并没有实现集合的修改方法。Arrays.asList 体现的是适配器模式，只是转换接口，后台的数据仍是数组。

```
String[] str = new String[] { "you", "wu" };
```

```
List list = Arrays.asList(str);
```

第一种情况：list.add("yangguanbao"); 运行时异常。

第二种情况：str[0] = "gujin"; 那么 list.get(0)也会随之修改。

5. 【强制】不要在 foreach 循环里进行元素的 remove/add 操作。remove 元素请使用 Iterator 方式，如果并发操作，需要对 Iterator 对象加锁。

6. 【推荐】集合初始化时，指定集合初始值大小。

说明：HashMap 使用 HashMap(int initialCapacity) 初始化，

7. 【推荐】使用 entrySet 遍历 Map 类集合 KV，而不是 keySet 方式进行遍历。

说明：keySet 其实是遍历了 2 次，一次是转为 Iterator 对象，另一次是从 hashMap 中取出 key 所对应的 value。而 entrySet 只是遍历了一次就把 key 和 value 都放到了 entry 中，效率更高。如果是 JDK8，使用 Map.forEach 方法。

正例：values()返回的是 V 值集合，是一个 list 集合对象；keySet()返回的是 K 值集合，是一个 Set 集合对象；entrySet()返回的是 K-V 值组合集合。

8. 【推荐】高度注意 Map 类集合 K/V 能不能存储 null 值的情况，如下表格：

| 集合类               | Key       | Value     | Super       | 说明               |
|-------------------|-----------|-----------|-------------|------------------|
| Hashtable         | 不允许为 null | 不允许为 null | Dictionary  | 线程安全             |
| ConcurrentHashMap | 不允许为 null | 不允许为 null | AbstractMap | 锁分段技术 (JDK8:CAS) |
| TreeMap           | 不允许为 null | 允许为 null  | AbstractMap | 线程不安全            |

|         |          |          |             |       |
|---------|----------|----------|-------------|-------|
| HashMap | 允许为 null | 允许为 null | AbstractMap | 线程不安全 |
|---------|----------|----------|-------------|-------|

反例：由于 HashMap 的干扰，很多人认为 ConcurrentHashMap 是可以置入 null 值，而事实上，存储 null 值时会抛出 NPE 异常。

## (六)并发处理

1. 【强制】获取单例对象需要保证线程安全，其中的方法也要保证线程安全。
2. 【强制】创建线程或线程池时请指定有意义的线程名称，方便出错时回溯。

正例：

```
public class TimerTaskThread extends Thread {
    public TimerTaskThread() {
        super.setName("TimerTaskThread");
        ...
    }
}
```

3. 【强制】线程资源必须通过线程池提供，不允许在应用中自行显式创建线程。

说明：使用线程池的好处是减少在创建和销毁线程上所花的时间以及系统资源的开销，解决资源不足的问题。如果不使用线程池，有可能造成系统创建大量同类线程而导致消耗完内存或者“过度切换”的问题。

4. 【强制】线程池不允许使用 Executors 去创建，而是通过 ThreadPoolExecutor 的方式，这样的处理方式让写的同学更加明确线程池的运行规则，规避资源耗尽的风险。

5. 【强制】SimpleDateFormat 是线程不安全的类，一般不要定义为 static 变量，如果定义为 static，必须加锁，或者使用 DateUtils 工具类。

说明：如果是 JDK8 的应用，可以使用 Instant 代替 Date，LocalDateTime 代替 Calendar，DateTimeFormatter 代替 SimpleDateFormat，官方给出的解释：simple beautiful strong immutable thread-safe。

6. 【强制】对多个资源、数据库表、对象同时加锁时，需要保持一致的加锁顺序，否则可能会造成死锁。

说明：线程一需要对表 A、B、C 依次全部加锁后才可以进行更新操作，那么线程二的加锁顺序也必须是 A、B、C，否则可能出现死锁。

7. 【强制】并发修改同一记录时，避免更新丢失，需要加锁。要么在应用层加锁，要么在缓存加锁，要么在数据库层使用乐观锁，使用 version 作为更新依据。

## (七)控制语句

1. 【强制】在一个 `switch` 块内，每个 `case` 要么通过 `break/return` 等来终止，要么注释说明程序将继续执行到哪一个 `case` 为止；在一个 `switch` 块内，都必须包含一个 `default` 语句并且放在最后，即使空代码。
2. 【强制】在 `if/else/for/while/do` 语句中必须使用大括号。即使只有一行代码。
3. 【推荐】循环体内的对象创建、变量定义、获取数据库连接、不必要的 `try-catch` 尽量移到循环体外。
4. 【推荐】除常用方法（如 `getXxx/isXxx`）等外，不要在条件判断中执行其它复杂的语句，将复杂逻辑判断的结果赋值给一个有意义的布尔变量名，以提高可读性。
6. 【推荐】避免采用取反逻辑运算符。
7. 【推荐】接口入参保护，这种场景常见的是用作批量操作的接口。
8. 【参考】调用频次低、执行开销大、稳定性要求高、对外开放、敏感权限入口的方法需做参数校验；极可能被循环调用、底层调用频度高、仅内部调用的方法可省略，但需在方法说明中注明外部检查要求。
9. 【参考】下列情形，不需要进行参数校验：
  - 1) 极有可能被循环调用的方法。但在方法说明里必须注明外部参数检查要求。
  - 2) 底层调用频度比较高的方法。毕竟是像纯净水过滤的最后一道，参数错误不太可能到底层才会暴露问题。一般 `DAO` 层与 `Service` 层都在同一个应用中，部署在同一台服务器中，所以 `DAO` 的参数校验，可以省略。
  - 3) 被声明成 `private` 只会被自己代码所调用的方法，如果能够确定调用方法的代码传入参数已经做过检查或者肯定不会有问题，此时可以不校验参数。

## (八)注释规约

1. 【强制】类、类属性、类方法的注释必须使用 `Javadoc` 规范，使用 `/**内容*/` 格式，不得使用 `//xxx` 方式。
2. 【强制】所有的抽象方法（包括接口中的方法）必须要用 `Javadoc` 注释、除了返回值、参数、异常说明外，还必须指出该方法做什么事情，实现什么功能。
3. 【强制】方法内部单行注释，在被注释语句上方另起一行，使用 `//` 注释。方法内部多行注释使用 `/* */` 注释，注意与代码对齐。

4. 【强制】所有的枚举类型字段必须要有注释，说明每个数据项的用途。
5. 【推荐】与其“半吊子”英文来注释，不如用中文注释把问题说清楚。专有名词与关键字保持英文原文即可。
6. 【推荐】代码修改的同时，注释也要进行相应的修改，尤其是参数、返回值、异常、核心逻辑等的修改。
7. 【参考】谨慎注释掉代码。在上方详细说明，而不是简单地注释掉。如果无用，则删除。
8. 【参考】对于注释的要求：第一、能够准确反应设计思想和代码逻辑；第二、能够描述业务含义，使别的程序员能够迅速了解到代码背后的信息。完全没有注释的大段代码对于阅读者形同天书，注释是给自己看的，即使隔很长时间，也能清晰理解当时的思路；注释也是给继任者看的，使其能够快速接替自己的工作。
9. 【参考】好的命名、代码结构是自解释的，注释力求精简准确、表达到位。避免出现注释的一个极端：过多过滥的注释，代码的逻辑一旦修改，修改注释是相当大的负担。

## 二、异常日志

### (一)异常处理

1. 【强制】Java 类库中定义的可以通过预检查方式规避的 `RuntimeException` 异常不应该通过 `catch` 的方式来处理，比如：`NullPointerException`，`IndexOutOfBoundsException` 等等。说明：无法通过预检查的异常除外，比如，在解析字符串形式的数字时，不得不通过 `catch` `NumberFormatException` 来实现。
2. 【强制】异常不要用来做流程控制，条件控制。
3. 【强制】`catch` 时请分清稳定代码和非稳定代码，稳定代码指的是无论如何不会出错的代码。对于非稳定代码的 `catch` 尽可能进行区分异常类型，再做对应的异常处理。
4. 【强制】捕获异常是为了处理它，不要捕获了却什么都不处理而抛弃之，如果不想处理它，请将该异常抛给它的调用者。最外层的业务使用者，必须处理异常，将其转化为用户可以理解的内容。
5. 【强制】有 `try` 块放到了事务代码中，`catch` 异常后，如果需要回滚事务，一定要注意手动回滚事务。
6. 【强制】`finally` 块必须对资源对象、流对象进行关闭，有异常也要做 `try-catch`。
7. 【强制】不要在 `finally` 块中使用 `return`。

8. 【强制】捕获异常与抛异常，必须是完全匹配，或者捕获异常是抛异常的父类。
9. 【推荐】方法的返回值可以为 null，不强制返回空集合，或者空对象等，必须添加注释充分说明什么情况下会返回 null 值。
10. 【推荐】防止 NPE 场景：基本类型接收包装类型返回值、数据库查询结果、集合中元素、远程调用返回、Session 取值、级联调用 obj.getA().getB().getC()。推荐使用 JDK8 Optional。
11. 【推荐】定义时区分 unchecked / checked 异常，避免直接抛出 new RuntimeException()，更不允许抛出 Exception 或者 Throwable，应使用有业务含义的自定义异常。推荐业界已定义过的自定义异常，如：DAOException / ServiceException 等。
12. 【参考】对外 HTTP/API 接口使用统一错误码；应用内部推荐异常抛出；跨服务调用优先使用统一 Result 封装（含 isSuccess()、错误码、错误信息）。
13. 遵循 DRY 原则，抽取重复的参数校验等公共逻辑为公共方法。

## (二) 日志规约

1. 【强制】应用中不可直接使用日志系统（Log4j、Logback）中的 API，而应依赖使用日志框架 SLF4J 中的 API，使用门面模式的日志框架，有利于维护和各个类的日志处理方式统一。
2. 【强制】日志文件推荐至少保存 15 天，因为有些异常具备以“周”为频次发生的特点。
3. 【强制】扩展日志命名 appName\_logType\_logName.log，如 digitalhuman\_monitor\_llmCost.log；错误日志与业务日志分开存放。  
digitalhuman\_llmCost\_llm.log —— 大模型调用（prompt 摘要、token 消耗、耗时、成本）  
digitalhuman\_auth\_auth.log —— 统一身份认证、人脸核验  
digitalhuman\_qa\_qa.log —— 智能问答与多轮对话  
digitalhuman\_pay\_pay.log —— 缴费、退款  
digitalhuman\_nav\_nav.log —— 校园导航与地标讲解
4. 【强制】对 trace/debug/info 级别的日志输出，必须使用条件输出形式或者使用占位符的方式。
5. 【强制】避免重复打印日志，浪费磁盘空间，务必在 log4j.xml 中设置 additivity=false。
6. 【强制】异常信息应该包括两类信息：案发现场信息和异常堆栈信息。如果不处理，那么通过关键字 throws 往上抛出。
7. 【推荐】谨慎地记录日志。生产环境禁止输出 debug 日志；有选择地输出 info 日志；如果使用 warn 来记录刚上线时的业务行为信息，一定要注意日志输出量的问题，避免把服务器磁盘撑

爆，并记得及时删除这些观察日志。

8. 【强制】日志中禁止输出身份证号、学号、手机号、人脸特征等敏感个人信息；确需记录时必须脱敏（如 158\\*\\*\\*9119），并统一脱敏规则。人脸核验、身份认证链路不得将证件信息写入日志。

## 三、安全规约

1. 【强制】隶属于用户个人的页面或者功能必须进行权限控制校验。

说明：防止没有做水平权限校验就可随意访问、修改、删除别人的数据，比如查看他人的私信内容、修改他人的订单。

2. 【强制】用户敏感数据禁止直接展示，必须对展示数据进行脱敏。

说明：个人手机号码显示为:158\*\*\*\*9119，隐藏中间 4 位，防止隐私泄露。

3. 【强制】用户输入的 SQL 参数严格使用参数绑定或者 METADATA 字段值限定，防止 SQL 注入，禁止字符串拼接 SQL 访问数据库。

4. 【强制】用户请求传入的任何参数必须做有效性验证。

说明：忽略参数校验可能导致：

| page size 过大导致内存溢出

| 恶意 order by 导致数据库慢查询

| 任意重定向

| SQL 注入

| 反序列化注入

| 正则输入源串拒绝服务 ReDoS

说明：Java 代码用正则来验证客户端的输入，有些正则写法验证普通用户输入没有问题，但是如果攻击人员使用的是特殊构造的字符串来验证，有可能导致死循环的结果。

5. 【强制】学生身份证号、学号、手机号、人脸特征等个人信息必须加密存储，展示时脱敏；人脸等敏感个人信息须单独取得学生授权，遵循最小必要原则，仅采集业务必需字段。（依据：《个人信息保护法》）

6. 【强制】数据库密码、API 密钥、公众号 AppSecret 等禁止硬编码在代码中，统一走配置中心或环境变量管理；密钥定期轮换。

7. 【强制】缴费、支付回调接口必须验签；金额以服务端计算为准，接口必须幂等，防止重复扣费与金额篡改。
8. 【强制】涉及学生个人数据的接口必须做水平越权防护：学生仅可查询/操作本人数据（本人报到进度、缴费记录、材料），禁止通过遍历 ID 访问他人数据；管理后台按学工、后勤、保卫、学院分权。
9. 【强制】学生上传的材料文件（证明材料、申请表、照片等）须校验文件类型、大小与内容，防恶意上传与脚本/XSS 文件。

## 四、MySQL 数据库

### (一)建表规约

1. 【强制】表达是与否概念的字段，必须使用 `is_xxx` 的方式命名，数据类型是 `unsigned tinyint`（1 表示是，0 表示否）。
2. 【强制】表名、字段名必须使用小写字母或数字，禁止出现数字开头，禁止两个下划线中间只出现数字。数据库字段名的修改代价很大，因为无法进行预发布，所以字段名称需要慎重考虑。说明：MySQL 在 Windows 下不区分大小写，但在 Linux 下默认是区分大小写。因此，数据库名、表名、字段名，都不允许出现任何大写字母，避免节外生枝。
3. 【强制】表名不使用复数名词。
4. 【强制】禁用保留字，如 `desc`、`range`、`match`、`delayed` 等，请参考 MySQL 官方保留字。
5. 【强制】主键索引名为 `pk_字段名`；唯一索引名为 `uk_字段名`；普通索引名则为 `idx_字段名`。
6. 【强制】小数类型为 `decimal`，禁止使用 `float` 和 `double`。
7. 【强制】长度几乎相等字符串使用 `char`。
8. 【强制】`varchar` 不超过 5000，超过了用 `text` 单独分表
9. 【强制】表必备三字段：`id`，`gmt_create`，`gmt_modified`。
10. 【推荐】表的命名最好是加上“业务名称\_表的作用”。
11. 【推荐】库名与应用名称尽量一致。
12. 【推荐】如果修改字段含义或对字段表示的状态追加时，需要及时更新字段注释。

## (二)索引规约

1. 【强制】业务上具有唯一特性的字段，即使是多个字段的组合，也必须建成唯一索引。
2. 【强制】超过三个表禁止 join。需要 join 的字段，数据类型必须绝对一致；多表关联查询时，保证被关联的字段需要有索引。
3. 【强制】在 varchar 字段上建立索引时，必须指定索引长度，没必要对全字段建立索引，根据实际文本区分度决定索引长度即可。
4. 【强制】页面搜索严禁左模糊或者全模糊，如果需要请走搜索引擎来解决。
5. 【推荐】防止因字段类型不同造成的隐式转换，导致索引失效。

## (三)SQL 语句

1. 【强制】不要使用 count(列名)或 count(常量)来替代 count(\*)，count(\*)是 SQL92 定义的标准统计行数的语法，跟数据库无关，跟 NULL 和非 NULL 无关。
2. 【强制】当某一列的值全是 NULL 时，count(col)的返回结果为 0，统计金额用 IFNULL 兜底  
正例：可以使用如下方式来避免 sum 的 NPE 问题：SELECT IF(ISNULL(SUM(g)),0,SUM(g))  
FROM table;
3. 【强制】使用 ISNULL()来判断是否为 NULL 值。  
说明：NULL 与任何值的直接比较都为 NULL。
4. 【强制】在代码中写分页查询逻辑时，若 count 为 0 应直接返回，避免执行后面的分页语句。
5. 【强制】不得使用外键与级联，一切外键概念必须在应用层解决。
6. 【强制】数据订正（特别是删除、修改记录操作）时，要先 select，避免出现误删除，确认无误才能执行更新语句。

## (四)ORM 映射

1. 【强制】在表查询中，一律不要使用 \* 作为查询的字段列表，需要哪些字段必须明确写明。  
说明：1) 增加查询分析器解析成本。2) 增减字段容易与 resultMap 配置不一致。
2. 【强制】POJO 类的布尔属性不能加 is，而数据库字段必须加 is\_，要求在 resultMap 中进行字段与属性之间的映射。  
说明：参见定义 POJO 类以及数据库字段定义规定，在<resultMap>中增加映射，是必须的。

在 MyBatis Generator 生成的代码中，需要进行对应的修改。

3. 【强制】查询结果必须映射为明确的 DO / VO / DTO 对象，禁止使用 Map、Hashtable 或 resultClass 作为查询结果集的输出。

说明：

1) 使用 Map / Hashtable 输出时，字段值的类型不可控，取值需要强转，容易引发 ClassCastException，且 IDE 无法对字段名做检查，拼错字段名要到运行时才暴露。

2) 使用 resultClass 使结果与类属性强耦合，表结构增减字段时容易与 Mapper 配置不一致，维护成本高。

3) 为每个查询定义明确的 resultMap 或对应的 DO / VO 类，使字段与业务对象解耦，方便复用和维护。

4. 【强制】sql.xml 配置参数使用：#{}, #param# 不要使用\${} 此种方式容易出现 SQL 注入。

5. 【强制】更新数据表记录时，必须同时更新记录对应的 gmt\_modified 字段值为当前时间。

6. 【推荐】不要写一个大而全的数据更新接口。传入为 POJO 类，不管是不是自己的目标更新字段，都进行 update table set c1=value1,c2=value2,c3=value3; 这是不对的。

## 五、工程结构

### 应用分层

1. 【推荐】图中默认上层依赖于下层，箭头关系表示可直接依赖，如：开放接口层可以依赖于 Web 层，也可以直接依赖于 Service 层，依此类推：

- 开放接口层：可直接封装 Service 方法暴露成 RPC 接口；通过 Web 封装成 http 接口；进行网关安全控制、流量控制等。
- 终端显示层：各个端的模板渲染并执行显示的层。当前主要是 velocity 渲染，JS 渲染，JSP 渲染，移动端展示等。
- Web 层：主要是对访问控制进行转发，各类基本参数校验，或者不复用的业务简单处理等。
- Service 层：相对具体的业务逻辑服务层。
- Manager 层：通用业务处理层，它有如下特征：

1) 对第三方平台封装的层，预处理返回结果及转化异常信息；

2) 对 Service 层通用能力的下沉，如缓存方案、中间件通用处理；

3) 与 DAO 层交互，对多个 DAO 的组合复用。

- DAO 层：数据访问层，与底层 MySQL、Oracle、Hbase 等进行数据交互。
- 外部接口或第三方平台：包括其它部门 RPC 开放接口，基础平台，其它公司的 HTTP 接口。

2. 【参考】（分层异常处理规约）在 DAO 层，产生的异常类型有很多，无法用细粒度的异常进行 catch，使用 catch(Exception e)方式，并 throw new DAOException(e)，不需要打印日志，因为日志在 Manager/Service 层一定需要捕获并打印到日志文件中，如果同台服务器再打日志，浪费性能和存储。在 Service 层出现异常时，必须记录出错日志到磁盘，尽可能带上参数信息，相当于保护案发现场。如果 Manager 层与 Service 同机部署，日志方式与 DAO 层处理一致，如果是单独部署，则采用与 Service 一致的处理方式。Web 层绝不应该继续往上抛异常，因为已经处于顶层，如果意识到这个异常将导致页面无法正常渲染，那么就应该直接跳转到友好错误页面，加上用户容易理解的错误提示信息。开放接口层要将异常处理成错误码和错误信息方式返回。

3. 【参考】分层领域模型规约：

- DO (Data Object)：与数据库表结构一一对应，通过 DAO 层向上传输数据源对象。
- DTO (Data Transfer Object)：数据传输对象，Service 或 Manager 向外传输的对象。
- BO (Business Object)：业务对象。由 Service 层输出的封装业务逻辑的对象。
- AO (Application Object)：应用对象。在 Web 层与 Service 层之间抽象的复用对象模型，极为贴近展示层，复用度不高。
- VO (View Object)：显示层对象，通常是 Web 向模板渲染引擎层传输的对象。
- Query：数据查询对象，各层接收上层的查询请求。注意超过 2 个参数的查询封装，禁止使用 Map 类来传输。

## 七、微服务与接口规约

1. 【强制】对外 RESTful 接口统一响应结构 code / message / data；错误码分段定义
2. 【强制】写操作接口必须幂等（缴费、提交材料、活动报名等）：使用幂等键或唯一索引，防止重复提交。
3. 【强制】分页参数必须校验：pageSize 设置上限，页码不能为负。
4. 【强制】WebSocket（数字人实时对话）连接必须鉴权（token 校验）；约定心跳机制、消息统一 JSON 格式（type / data / msgId）、断线重连与消息去重。

## 八、AI 与大模型应用规约

1. 【强制】数字人问答的用户输入与模型输出必须经过敏感词 / 内容安全过滤，违规内容拦截并给出规范提示，不得原样回显。
2. 【强制】若调用大模型：送入模型的用户数据必须脱敏（学号、身份证、手机号不得原样进入 prompt）；LLM 调用记录 token 消耗与成本日志（digitalhuman\_llmCost\_llm.log）。若未接入大模型，本条不适用。
3. 【强制】多轮对话设置超时兜底：超时或异常时给用户友好提示，不暴露内部错误；模型不可用时降级到固定问答库或提示转人工。
4. 【推荐】数字人语音交互控制并发连接数，超限时排队或提示稍后再试。

---

i